ECE 315 Lab 3: Zynq-7000 SPI Interface

Authors: Yaaqoob Choulli, Rowan Rasmusson

Student ID: 1752634, 1757167

CCID: yaaqoob, rrasmuss

Course: ECE 315

Lab Section: H21

Date Performed: March 10, 2026

Date Due: March 24, 2026

Abstract:

The goal of the third lab in computer interfacing is to gain practical experience with the Serial Peripheral Interface (SPI) protocol in both controller (main) and peripheral (sub) modes, as well as to delve into embedded graphics programming using an SPI interface. The hardware utilized for these exercises was the Digilent Zybo Z7 development board featuring a Zynq-7000 processor, alongside the Xilinx/AMD Pmod OLED (revision A) module. In the first part of the lab, we successfully implemented a polled-mode SPI loopback system where CPU0 ran a FreeRTOS real-time kernel managing serial UART communications and SPI data transfers between the SPI0 and SPI1 hardware nodes. In the second part, we applied our knowledge to design a real-time embedded "Pong" game, decoupling input polling, game physics, and OLED rendering into separate FreeRTOS tasks to achieve a robust and interactive system.

Design:

Part 1: Implementing SPI Loopback

The objective for Part 1 was to establish a communication loop between a host PC serial terminal, the UART1 interface, and the SPI0/SPI1 interfaces. The FreeRTOS application was divided into four distinct tasks: vUartManagerTask, vSpiMainTask, vSpiSubTask, and vRgbLedTask.

When the user types into the terminal, vUartManagerTask reads the bytes. Depending on the loopback state (toggled via <ENTER>1<ENTER> or <ENTER>2<ENTER>), the bytes are either echoed locally or forwarded to the uart_to_spi queue, depending on whether <ENTER>1<ENTER> has been pressed or not. When SPI loopback is active, vSpiMainTask receives these bytes, loads them into a transmit frame, and uses our custom spiMasterTransfer() function (written in my_spi.c) to push the data over the hardware SPI lines to the SPI1 peripheral.

The vSpiSubTask acts as the slave device. It receives the incoming data and mirrors it back to the master. Additionally, we implemented logic within this task to continuously monitor the incoming byte stream for a termination sequence (\r%\r). Upon detecting this sequence, vSpiSubTask calculates the total bytes received and the total number of messages, formatting this data into a string using snprintf(). This report string is then chunked and sent back over the SPI lines to the master, which routes it through the spi_to_uart queue for display on the terminal.

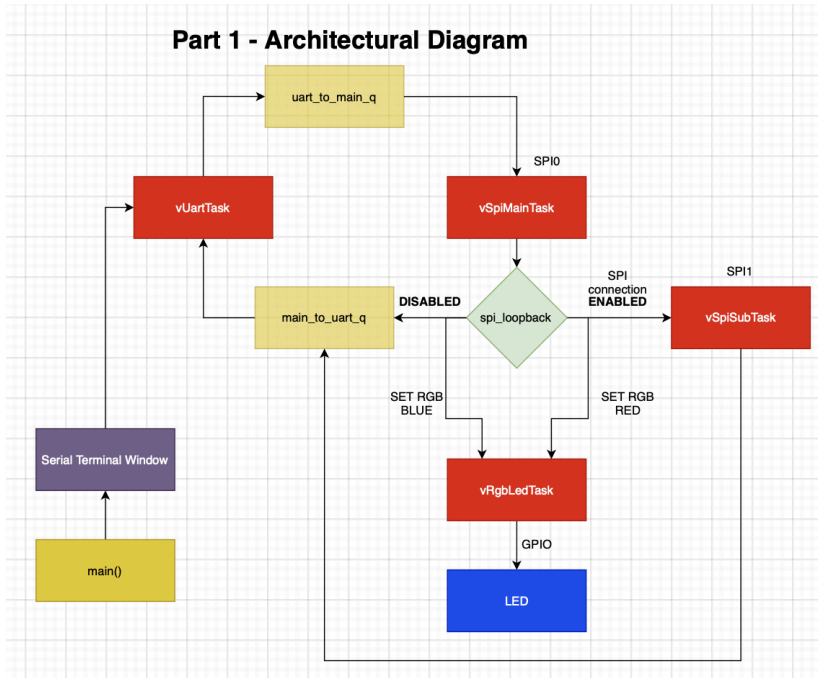

Figure 1: Initial Architectural Diagram. This diagram illustrates the UART layer routing data through the queues to the SPI tasks, highlighting the physical loopback between SPI0 and SPI1. This picture comes from the prelab.

```


void spiMasterTransfer(const u8 *tx, u8 *rx, int byteCount)
{
    // TODO 6: write the body for this function using spiMasterWrite and spiMasterRead
    spiMasterWrite(tx, byteCount);
    vTaskDelay(10);
    spiMasterRead(rx, byteCount);
}


```

Figure 2: This code snippet shows the implementation of the spiMasterTransfer function. Since one can think of SPI as a large shift register, it is meant to write a value, then read another, so that when the value is read from this instance again, it gets the value that it wrote beforehand.

```
void spiSlaveTransfer(const u8 *tx, u8 *rx, int byteCount)
{
    spiSlaveWrite(tx, byteCount);
    spiSlaveRead(rx, byteCount);
}
```

Figure 3: This is the slave side of the transfer. It is intended to achieve the same goal as in the master transfer function described above.

Part2: SPI OLED Screen Interface - Embedded Game

For Part 2, we were tasked with developing an interactive application utilizing the Pmod OLED screen and at least three input peripherals. We designed an embedded version of the classic game "Pong". To adhere to best architectural practices, we separated the logic into five distinct FreeRTOS tasks, communicating exclusively via queues:

1. **keypadTask**: Scans the keypad for user input. It sends paddle movement commands ('2' for up, '8' for down) to q_paddle, and difficulty selection commands ('4', '5', '6') to both q_difficulty_game and q_difficulty_rgb.
2. **gameTask**: The core physics and rendering engine. It computes ball trajectories, handles wall and paddle collisions, tracks the score, and manages the OLED frame updates using the custom OLED_ClearBuffer(), OLED_DrawLineTo(), and OLED_RectangleTo() functions.
3. **buttonTask**: Polls the physical pushbuttons. If a button is pressed when the player reaches zero lives, it sends a command via q_reset to restart the game.
4. **ssdTask**: Consumes the current life count from q_lives and multiplexes the 7-segment display to provide real-time numeric feedback to the user.
5. **rgbTask**: Consumes difficulty updates from q_difficulty_rgb and changes the RGB LED color (Green = Easy, Blue = Medium, Red = Hard) to provide visual status indication.

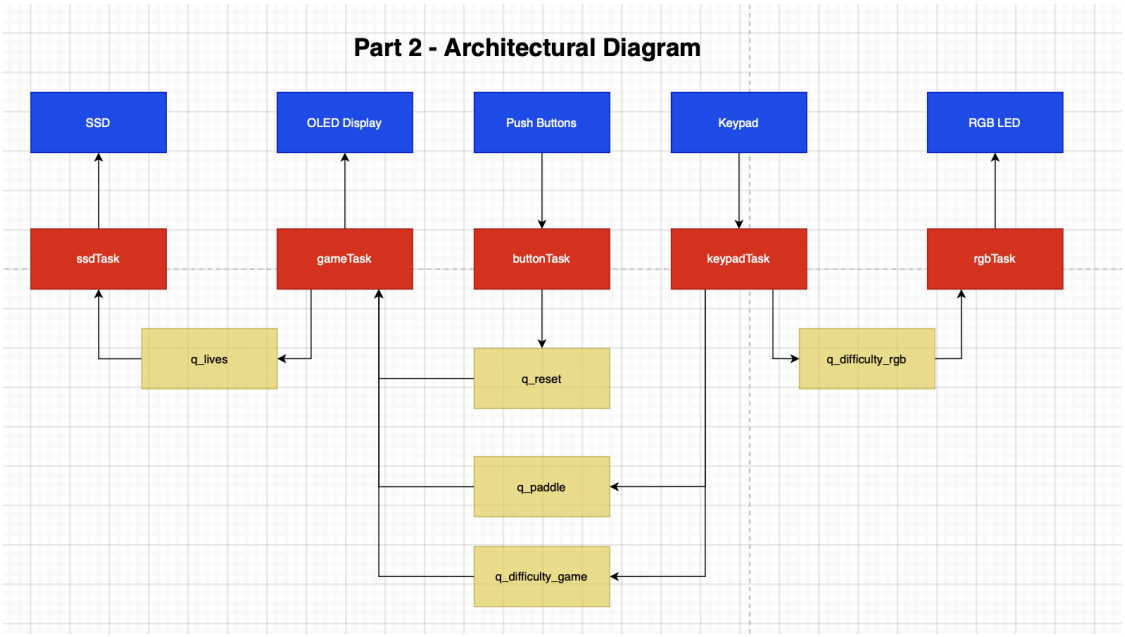

Figure 4: This diagram illustrates the multi-tasking architecture, highlighting that peripheral tasks serve as producers while the gameTask acts as the primary consumer.

<u>SPI Hardware Config from VIVADO</u>

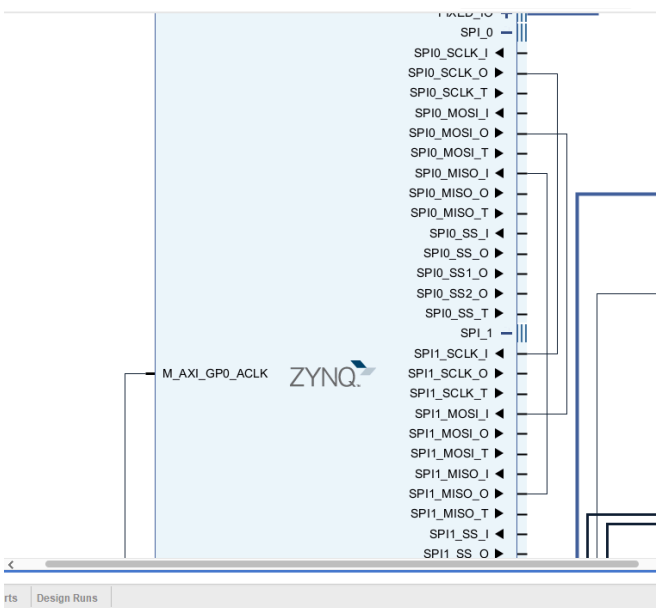

Figure 5: This shows the SPI configuration in the IP Config section of Vivado that was used to configure the Zybo. It shows a 3-wire implementation of SPI having the clocks, MOSI and MISO connected to each other for each instance. This is what allows spi_loopback for part 1.

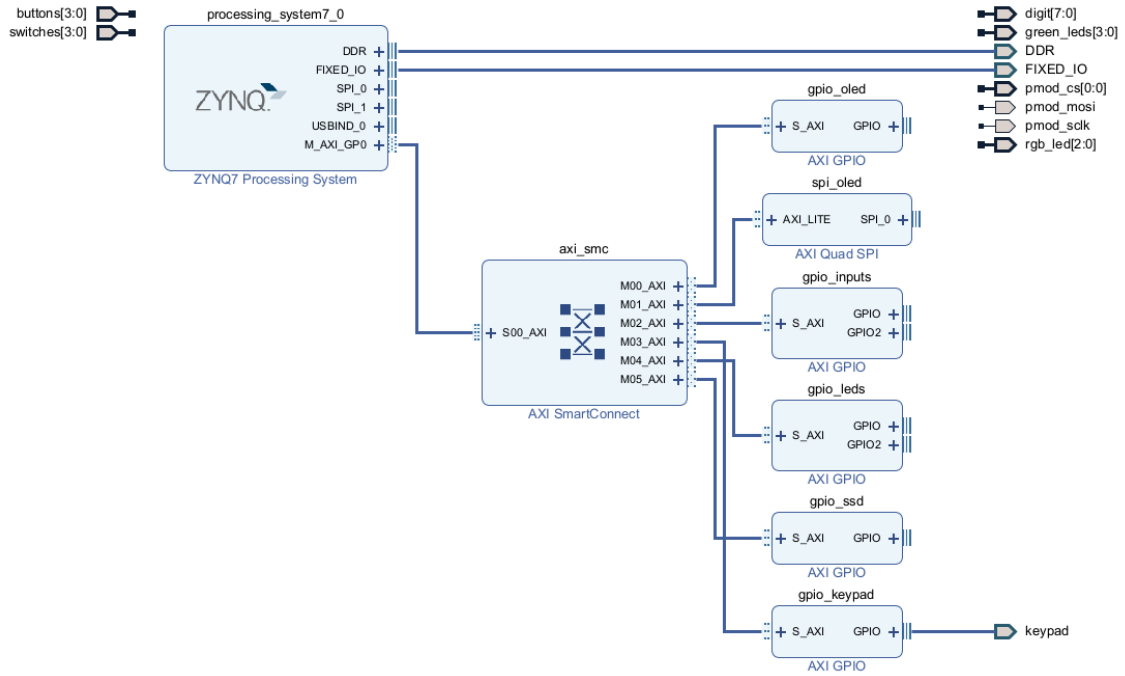

Figure 6: This shows the overall hardware block design in Vivado used to configure the Zybo board. It illustrates the ZYNQ7 Processing System connected through an AXI SmartConnect to various peripheral blocks, including the AXI Quad SPI and multiple AXI GPIO instances required to interface with the OLED, keypad, seven-segment display, LEDs, and physical buttons.

<u>Discussion:</u>

Part 1: During the implementation of Part 1, we encountered a few notable issues. First, completing TODO 14 in vUartManagerTask proved to be quite tricky. We initially had a hard time knowing how to call the terminateInput() function correctly. At one point, we were seeing infinite output but not the exit message. Realizing we needed to clear the report flag and safely exit the reporting state once the final $ control character was received from the spi_to_uart queue, we determined that we needed to call terminateInput(). This resolved all of these issues, clearing the necessary flags and safely printing the exit string.

Another significant issue was a bug in our transfer counting logic. Our message_byte_count and total_bytes_received_over_spi variables kept improperly incrementing by an extra 3 bytes per message. We realized the system was counting the \r%\r termination command sequence as standard SPI payload transfers. To resolve this, we added logic in vSpiSubTask to explicitly subtract 3 from the counters whenever the termination sequence was triggered, ensuring accurate reporting. This is seen on the next page.

```
report_len = snprintf(report,
                      sizeof(report),
                      "\nNumber of bytes received over SPI:%d\nLast message byte count: %d\nTotal messages received: %d\n",
                        total_bytes_received_over_spi - (3*total_messages_received), last_message_byte_count, total_messages_received);
```

Figure 7: The total bytes over SPI is linearly decremented by three times the number of messages received. This correctly removes the termination commands after every message.

A potential improvement for this part of the lab would be transitioning from a polled SPI driver to an interrupt-driven SPI driver. As we learned in Lab 2, polling requires continuous CPU involvement, whereas hardware interrupts would allow the system to react asynchronously to incoming data, freeing up the CPU for other tasks.

Part 2: The development of our Pong game was mostly smooth sailing, but we initially fell into a poor design pattern. To get the game running quickly, we defined the game state variables (such as paddle position, ball coordinates, score, and lives) as global variables accessed across multiple tasks.

Realizing this bypassed the safety and architectural decoupling provided by an RTOS, we decided to refactor the entire system to use FreeRTOS queues. This transition was surprisingly tricky and required some heavy debugging at certain points. For example, we ran into some synchronization issues between producer tasks (keypad/buttons) and the consumer tasks (game physics/displays). The solution that ultimately gave us success was stepping back and creating a detailed architectural diagram (shown in the Design section, Figure 5). Visualizing the components and the exact queue pipelines (q_paddle, q_difficulty_game, q_reset, etc.) made mapping out the inter-task communication straightforward and allowed us to finalize the game logic.

Testing Methodology:

For testing, we manually engaged with the hardware inputs to verify both the SPI loopback integrity (Part 1) and the game logic (Part 2).

Test	Expected Output	Rationale
Part 1: SPI Loopback: ‘<ENTER>2<ENTER>’ then type "hello"	Terminal displays "hello", RGB LED turns Green.	Verifies that the SPI loopback is enabled and vSpiSubTask successfully mirrors the payload over the physical SPI lines.
Part 1: Type **\r%\r** after entering characters “**hello**”.	Terminal outputs the formatted string showing exact byte count and message count. Byte count should be 5, and	Ensures the string matching logic correctly identifies the termination sequence and executes the formatting block.

	message count should be 1.	
Part 1: Uart Loopback: ‘<ENTER>1<ENTER>’ then type "test"	Terminal displays repeated characters of “test”. Data is not passed to SPI queues or tasks.	Verifies that activating the UART loopback correctly restricts data flow entirely to vUartManagerTask for local echoing, ensuring the SPI loopback is properly disabled.
Part 1: Multiple messages: ‘<ENTER>2<ENTER>’ then “**cat**”, followed by **\r%\r**. Then another message “**dogs**” followed by **\r%\r**.	The first summary output should have 1 message and 3 bytes. The second summary should have 2 messages and 7 bytes.	Ensures the counting logic accurately tracks multiple separate messages over the hardware SPI lines and correctly subtracts the 3 bytes of the **\r%\r** termination sequence from the final payload byte count.
Part 1: Enter <ENTER>2<ENTER> then immediately type **\r%\r**	Terminal outputs the formatted report string. The exact byte count should be 0.	Verifies the boundary condition where the **\r%\r** termination sequence is sent without any preceding payload data. It ensures your specific fix of subtracting 3 from the counters correctly handles an empty payload without resulting in a negative byte count.
Part 1: Enter <ENTER>2<ENTER> then type “**\r%hello**”	Terminal displays **\r%hello** but does not print the final report string.	Ensures the vSpiSubTask termination sequence detection strictly requires the full, exact **\r%\r** string to trigger the formatting block, correctly treating partial matches as standard SPI payload transfers.
Part 2: Press Keypad '8' repeatedly	The paddle moves downward on the OLED until it hits the bottom boundary and stops moving.	Verifies the q_paddle queue transmission and boundary clamping logic in the physics engine.
Part 2: Press Keypad '6'	RGB LED turns Red, ball velocity noticeably increases on OLED.	Ensures the q_difficulty multi-queue broadcast correctly updates both the visual indicator task and the physics engine task.

Part 2: Let ball pass paddle 3 times	SSD decrements from 3 to 0. OLED displays "Game Over" and final score.	Validates collision logic, inter-task communication to the ssdTask, and state-change logic to end the game loop.
Part 2: Press Pushbutton after Game Over	Game resets: Score to 0, Lives to 3, Ball and Paddle to default positions.	Ensures buttonTask can trigger a full state reset via the q_reset queue.

<u>Answers to Questions:</u>

1. Impact of Loopback Mode Activation on Task Interaction. Activating the UART loopback restricts data flow entirely to vUartManagerTask; characters are echoed back to the terminal without ever entering the queues. On the other hand, activating the SPI loopback fundamentally changes the data routing: vUartManagerTask stops local echoing and instead acts as a producer, pushing data to vSpiMainTask via the uart_to_spi queue. vSpiMainTask then dictates the hardware interaction, pushing data to vSpiSubTask over the physical SPI lines, retrieving the response, and placing it in the spi_to_uart queue for terminal output.

2. Difficulties Setting Up SPI Communication. A primary challenge in setting up the SPI communication was ensuring synchronization between the Master and Sub tasks. Initially, attempting to perform back-to-back writes and reads without adequate spacing resulted in dropped bytes or duplicated data. We resolved this by carefully structuring our spiMasterTransfer function to sequence the spiWrite and spiRead operations, and leveraging vTaskDelay to allow the hardware FIFOs time to process the shift registers without blocking the RTOS scheduler.

3. Challenges Interfacing with the Pmod OLED. Interfacing with the OLED screen introduced complexities regarding screen flickering and frame tearing. Because the OLED driver relies on SPI to push pixel data, sending excessive discrete drawing commands caused visual stuttering. To resolve this, we optimized our gameTask to clear the buffer (OLED_ClearBuffer), calculate all new positions (paddle and ball), draw all elements to the local memory, and finally issue a single OLED_Update command per frame cycle.

4. Dynamic Graphics and Peripheral Input Our application heavily relies on peripheral input to drive dynamic graphics. The keypadTask directly influences the Y-coordinate of the paddle drawn on the OLED screen. When the user presses '2' or '8', the paddle shifts up or down in real-time to intercept the ball. Furthermore, pressing the difficulty keys alters the base_speed_x and base_speed_y variables within the game engine, dynamically increasing the velocity at which the ball is rendered across the screen frame-by-frame.

5. Deepened Understanding of Embedded Systems and RTOS This lab significantly deepened our appreciation for task decoupling within a real-time OS framework. By utilizing queues to pass physics variables (like paddle movement) rather than sharing global state variables, we entirely avoided race

conditions. It also highlighted the importance of hardware abstraction; writing custom SPI wrappers allowed us to isolate low-level register manipulation from the high-level application logic.

Conclusion:

The primary objective of this lab was to gain practical experience utilizing the Serial Peripheral Interface (SPI) protocol in both controller (main) and peripheral (sub) modes, alongside exploring embedded graphics programming with the Pmod OLED module. We accomplished this in two distinct parts. First, we implemented a polled-mode SPI loopback system on the Zybo Z7 board, where a FreeRTOS real-time kernel managed UART communications and SPI data transfers between the SPI0 and SPI1 hardware nodes. Second, we designed a real-time embedded "Pong" game, successfully decoupling hardware input polling, game physics, and OLED rendering into five separate FreeRTOS tasks that communicated exclusively via queues.

Ultimately, the lab was a complete success, resulting in a stable SPI communication loop and a highly responsive, interactive embedded game. However, we encountered several significant challenges during development. For example in Part 1, we struggled to realize the use of terminateInput() to safely exit the reporting state. In Part 2, our initial game design relied heavily on global variables, which bypassed RTOS safety mechanisms thus we refactoring the system to use FreeRTOS queues which introduced tricky synchronization issues. However, these challenges were overcome and we were left with a better understanding for embedded programing and FreeRTOS synchronization architecture.

Given the success of the lab we did notice some potential improvements that could have been made. A key limitation of our final design is the reliance on a polled SPI driver, a major potential improvement would be migrating both the SPI communication and the peripheral inputs to an interrupt-driven architecture. As demonstrated in prior labs, utilizing hardware interrupts would entirely eliminate polling overhead, prevent any dropped inputs during rapid gameplay, and result in a significantly more efficient and robust embedded system. With this being said, all in all the lab was a big success.

References:

1. Provided lab documentation and source code
2. Inclass notes
3. FreeRTOS Documentation; https://freertos.org/Documentation/00-Overview
5. Made with help from Gemini Pro